UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

EBB 320

Control Systems

Practical 1: Report

Group 28

Name and Surname	Student Number	Signature	% Contribution
KC. Mattheus	17049271		33
A. van Niekerk	19007583		33
TS Fogwill	18321692		33

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

September 6, 2021

Contents

1 Introduction 2
1.1 Aim 2
2 Linearization 2
2.1 Task 1 2
3 Transfer functions 4
3.1 Task 2 4
4 Simulations 5
4.1 Task 3 5
4.2 Task 4 11
5 Conclusion 14

A Python Code **16**

List of Figures

4.1.1 Angular speed of left wheel and right wheel vs time using control input set 1 5
4.1.2 Nonlinear and linear X-positions vs time using control input set 1 6
4.1.3 Nonlinear and linear Y-positions vs time using control input set 1 6
4.1.4 Nonlinear and linear angular displacements vs time using control input set 1 7
4.1.5 Nonlinear and linear Y-positions vs X-positions using control input set 1 7
4.1.6 Angular speed of left wheel and right wheel vs time using control input set 2 8
4.1.7 Nonlinear and linear X-positions vs time using control input set 2 9
4.1.8 Nonlinear and linear Y-positions vs time using control input set 2 9
4.1.9 Nonlinear and linear angular displacements vs time using control input set 2 10
4.1.10 Nonlinear and linear Y-positions vs X-positions using control input set 2 10
4.2.1 Angular speed of left wheel and right wheel vs time 11
4.2.2 Nonlinear and linear X-positions vs time 12
4.2.3 Nonlinear and linear Y-positions vs time 12
4.2.4 Nonlinear and linear angular displacements vs time 13
4.2.5 Nonlinear and linear Y-positions vs X-positions 13

1 Introduction

In this Practical, a sensor array will be used to control the line tracking robot (MARV), which consists of multiple RGB color sensors and two motors. The MARV must be able to follow different colored lines, called tracks. The MARV sensor array and motor speed will be converted and expressed as a linear model. Using the sensor array as input, the error is calculated as a deviation parallel to the track.

Firstly, the nonlinear system is linearized around the equilibrium points. Then transfer functions of the state space model are derived. Through differential equations, Python will be used to simulate linear and non-linear systems using the given parameters with two random control inputs. Finally, the simulation is repeated using the given input and required control parameters.

In this practical the non-linear systems are linearized because it is easier to calculate and implement, it is also required to use the system in a state space. Due to the low complexity of the required system, the MARV control system can be linearized. In this practical, mechanisms specifically designed for linear systems, such as transfer functions and state space controllers, will be used.

1.1 Aim

The aim of this practical is to get a linear model of the MARV, we then explore the advantages and disadvantages of linear and non-linear systems, and study the effect of linearization of the system at specific equilibrium points.

2 Linearization

2.1 Task 1

For task one the non-linear equations of the MARV will be approximated using the process of linearization to obtain linear equations.
The non-linear representation for the MARV is given as:

$$f(t, \mathbf{z}, \mathbf{u}) = \dot{\mathbf{z}}(t) = \begin{bmatrix} \dot{z_1}(t) \\ \dot{z_2}(t) \\ \dot{z_3}(t) \end{bmatrix} = \begin{bmatrix} \frac{R}{2}(u_1(t) + u_2(t))cos(z_3(t)) \\ \frac{R}{2}(u_1(t) + u_2(t))sin(z_3(t)) \\ \frac{R}{L}(u_2(t) - u_1(t)) \end{bmatrix} \tag{1.1}$$

$$g(t, \mathbf{z}, \mathbf{u}) = y(t) = z_3(t) \tag{1.2}$$

with $f(t, \mathbf{z}, \mathbf{u})$ representing the state equations and $g(t, \mathbf{z}, \mathbf{u})$ representing the MARV's output.

The initial state conditions are given as:

$$\mathbf{z}(0) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{1.3}$$

All values are zero as the MARV will be placed centred and parallel to the track at the start of each race.

The equilibrium points $(\mathbf{z}_e, \mathbf{u}_e, y)$ are given as:

$$\mathbf{z}_e = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \mathbf{u}_e = \begin{bmatrix} \pi \\ \pi \end{bmatrix}, y_e = 0 \tag{1.4}$$

To use the MARV in a state space, a linear model is required. Thus the above equations will be used to linearize the model to be used in a state space. The MARV will be linearised about the equilibrium point $(\mathbf{z}_e, \mathbf{u}_e, y)$. The state space model can be represented as follows:

$$\Delta\dot{\mathbf{z}}(t) = \mathbf{A}\Delta\mathbf{z}(t) + \mathbf{B}\Delta\mathbf{u}(t) \tag{1.5}$$

$$\Delta y(t) = \mathbf{C}\Delta\mathbf{z}(t) \tag{1.6}$$

The state-space matrices A, B and C can be calculated as:

$$A = \left.\frac{\partial f}{\partial z^T}\right|_{z_e,u_e}, B = \left.\frac{\partial f}{\partial u^T}\right|_{z_e,u_e}, C = \left.\frac{\partial g}{\partial z^T}\right|_{z_e,u_e} \tag{1.7}$$

$$A = \left.\frac{\partial \dot{\mathbf{z}}}{\partial \mathbf{z}^T}\right|_{z_e,u_e} = \left.\begin{bmatrix} 0 & 0 & -\frac{R}{2}(u_1(t)+u_2(t))sin(z_3(t)) \\ 0 & 0 & \frac{R}{2}(u_1(t)+u_2(t))cos(z_3(t)) \\ 0 & 0 & 0 \end{bmatrix}\right|_{z_e,u_e} \tag{1.8}$$

$$\therefore A = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & R\times\pi \\ 0 & 0 & 0 \end{bmatrix} \tag{1.9}$$

$$B = \left.\frac{\partial \dot{\mathbf{z}}}{\partial \mathbf{u}^T}\right|_{z_e,u_e} = \left.\begin{bmatrix} \frac{R}{2}(cos(z_3(t))) & \frac{R}{2}(cos(z_3(t))) \\ \frac{R}{2}(sin(z_3(t))) & \frac{R}{2}(sin(z_3(t))) \\ -\frac{R}{L} & \frac{R}{L} \end{bmatrix}\right|_{z_e,u_e} \tag{1.10}$$

$$\therefore B = \begin{bmatrix} \frac{R}{2} & \frac{R}{2} \\ 0 & 0 \\ -\frac{R}{L} & \frac{R}{L} \end{bmatrix} \tag{1.11}$$

$$C = \left.\frac{\partial \dot{y}}{\partial \mathbf{z}^T}\right|_{z_e,u_e} = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}|_{z_e,u_e} \tag{1.12}$$

$$\therefore C = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \tag{1.13}$$

$$\therefore \Delta\dot{\mathbf{z}}(t) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & R\times\pi \\ 0 & 0 & 0 \end{bmatrix}\mathbf{z}(t) + \begin{bmatrix} \frac{R}{2} & \frac{R}{2} \\ 0 & 0 \\ -\frac{R}{L} & \frac{R}{L} \end{bmatrix}\mathbf{u}(t) \tag{1.14}$$

$$\therefore \mathbf{y} = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}\mathbf{z}(t) \tag{1.15}$$

3 Transfer functions

3.1 Task 2

The transfer functions $Z_3(s)/U_1(s)$ and $Z_3(s)/U_2(s)$ will be obtained from calculations seen below.

$$\dot{\mathbf{z}} = \mathbf{A}\mathbf{z} + \mathbf{z} \tag{1.16}$$

$$\mathbf{z} = \mathbf{B}\mathbf{u} \tag{1.17}$$

Therefore substituting equation 1.17 into equation 1.16 the below is obtained.

$$\mathbf{B}\mathbf{u} = \dot{\mathbf{z}} - \mathbf{A}\mathbf{z} \tag{1.18}$$

Transforming equation 1.18 into the Laplace domain the bellow is obtained.

$$L\{\mathbf{B}\mathbf{u}\} = L\{\dot{\mathbf{z}} - \mathbf{A}\mathbf{z}\} \tag{1.19}$$

$$\mathbf{B}U(s) = Z(s)s - \mathbf{A}Z(s) \tag{1.20}$$

$$\mathbf{B}U(s) = Z(s) \times (s\mathbf{I} - \mathbf{A}) \tag{1.21}$$

$$\frac{Z(s)}{U(s)} = (s\mathbf{I} - \mathbf{A})^{-1} \times \mathbf{B} \tag{1.22}$$

Using Z_3 and U_1 the transfer function $\frac{Z_3(s)}{U_1(s)}$ can be obtained as seen below:

$$\frac{Z_3(s)}{U_1(s)} = \frac{1}{s^3}\begin{bmatrix}0 & 0 & s^2\end{bmatrix}\begin{bmatrix}\frac{R}{2} \\ 0 \\ -\frac{R}{L}\end{bmatrix} \tag{1.23}$$

$$\therefore \frac{Z_3(s)}{U_1(s)} = -\frac{R}{Ls} \tag{1.24}$$

Using Z_3 and U_2 the transfer function $\frac{Z_3(s)}{U_2(s)}$ can be obtained as seen below:

$$\frac{Z_3(s)}{U_2(s)} = \frac{1}{s^3}\begin{bmatrix}0 & 0 & s^2\end{bmatrix}\begin{bmatrix}\frac{R}{2}\\ 0\\ \frac{R}{L}\end{bmatrix} \tag{1.25}$$

$$\therefore \frac{Z_3(s)}{U_2(s)} = \frac{R}{Ls} \tag{1.26}$$

The system of equations is linearized. To obtain the differential equations of the linear model the non-linear model must be evaluated at the operating point. The evaluated non-linear model then needs to be added to the system model, as shown below.

$$\dot{z}(t) = \mathbf{A}z(t) + \mathbf{B}u(t) + f(t, z_e, U_e) \tag{1.27}$$

4 Simulations

4.1 Task 3

Input 1

$u_L = sin(t) \times e^{-t}$

$u_R = \frac{t}{25}$

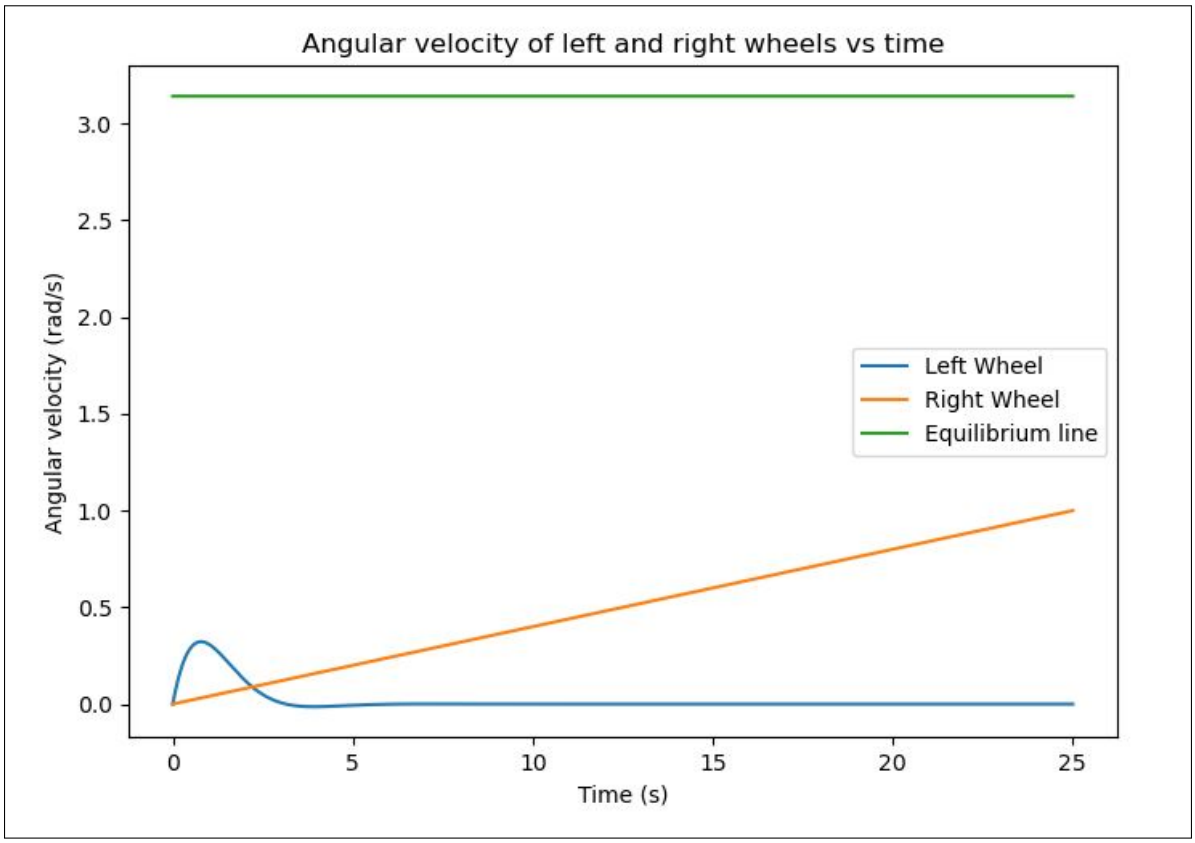

Figure 4.1.1: Angular speed of left wheel and right wheel vs time using control input set 1

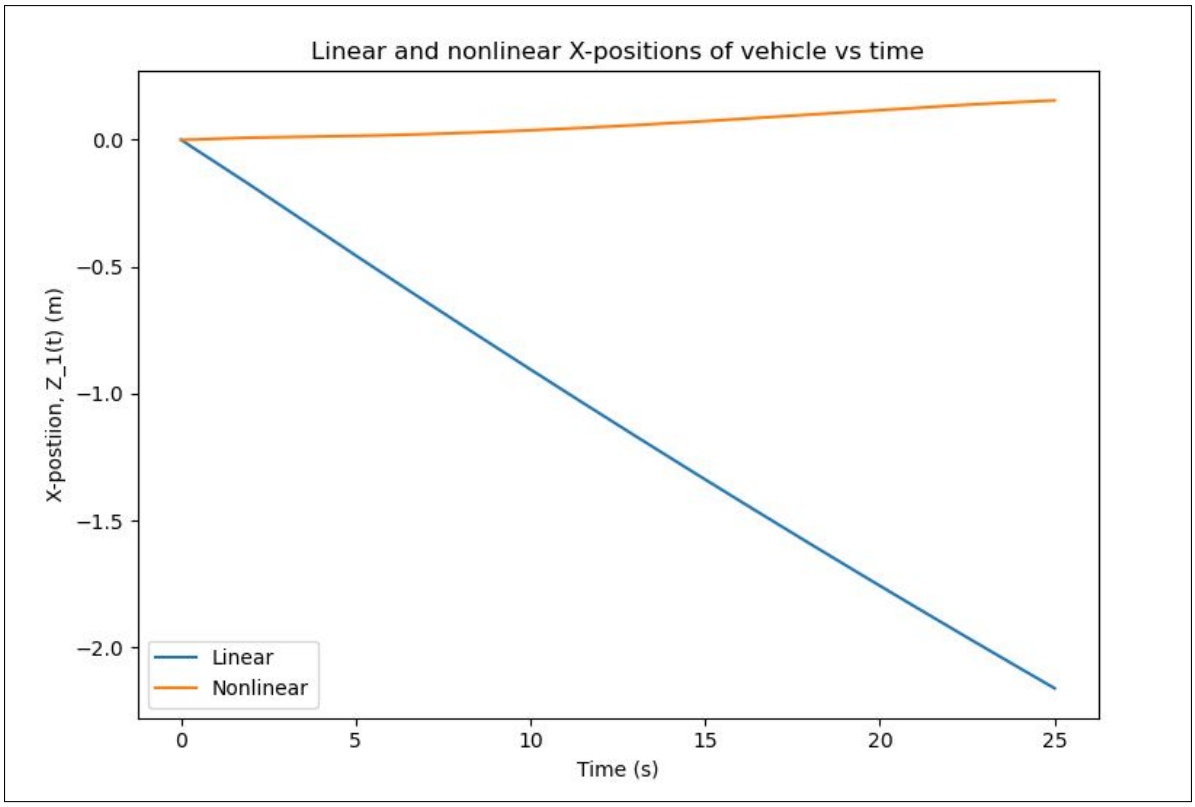

Figure 4.1.2: Nonlinear and linear X-positions vs time using control input set 1

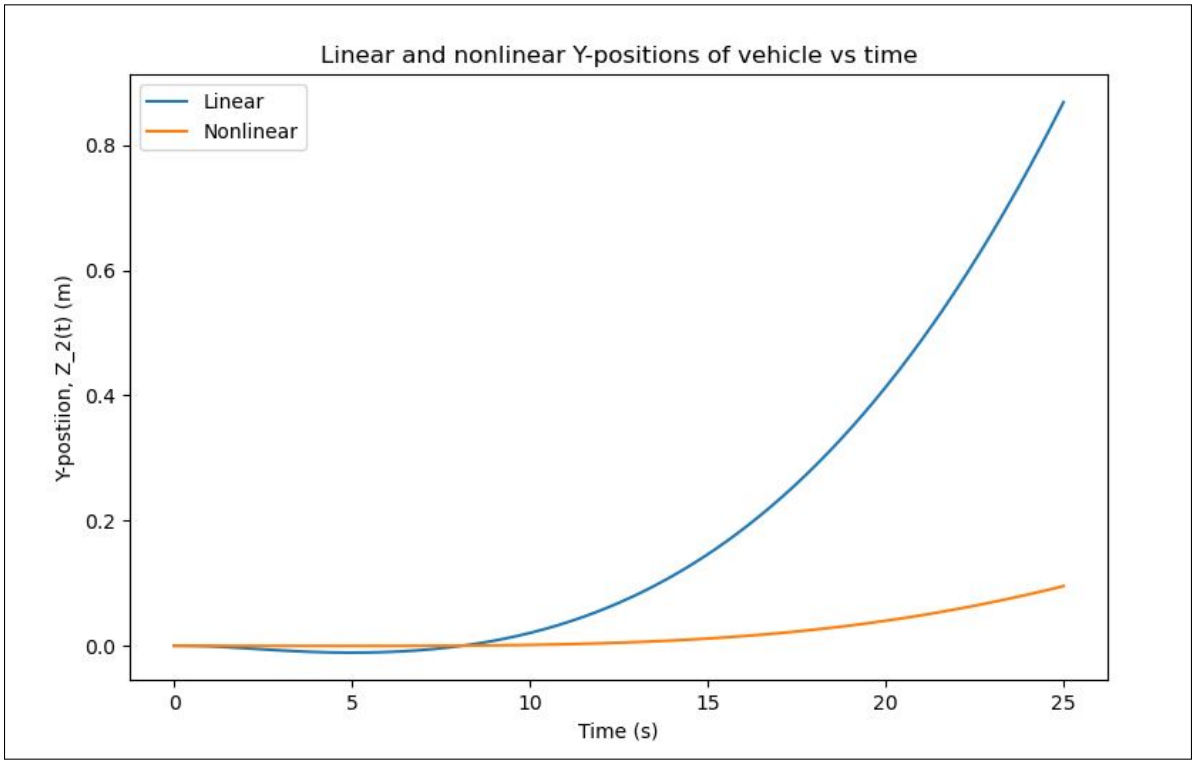

Figure 4.1.3: Nonlinear and linear Y-positions vs time using control input set 1

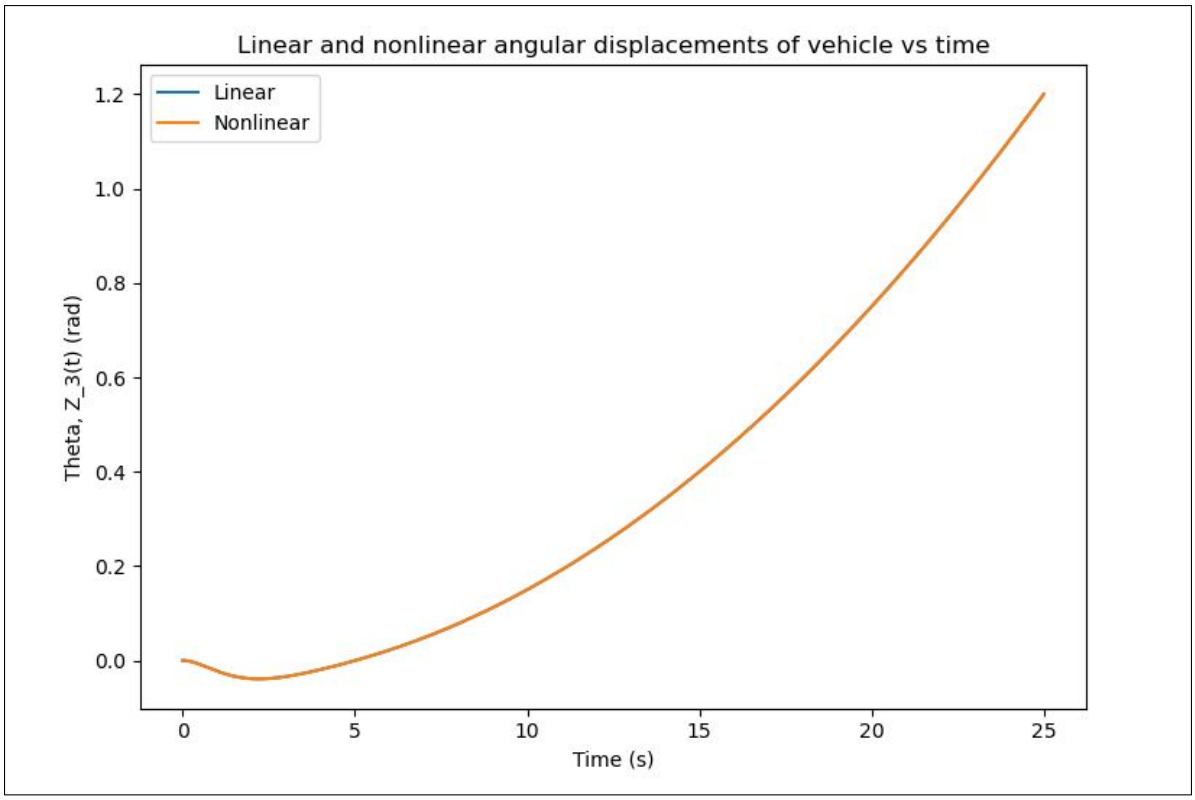

Figure 4.1.4: Nonlinear and linear angular displacements vs time using control input set 1

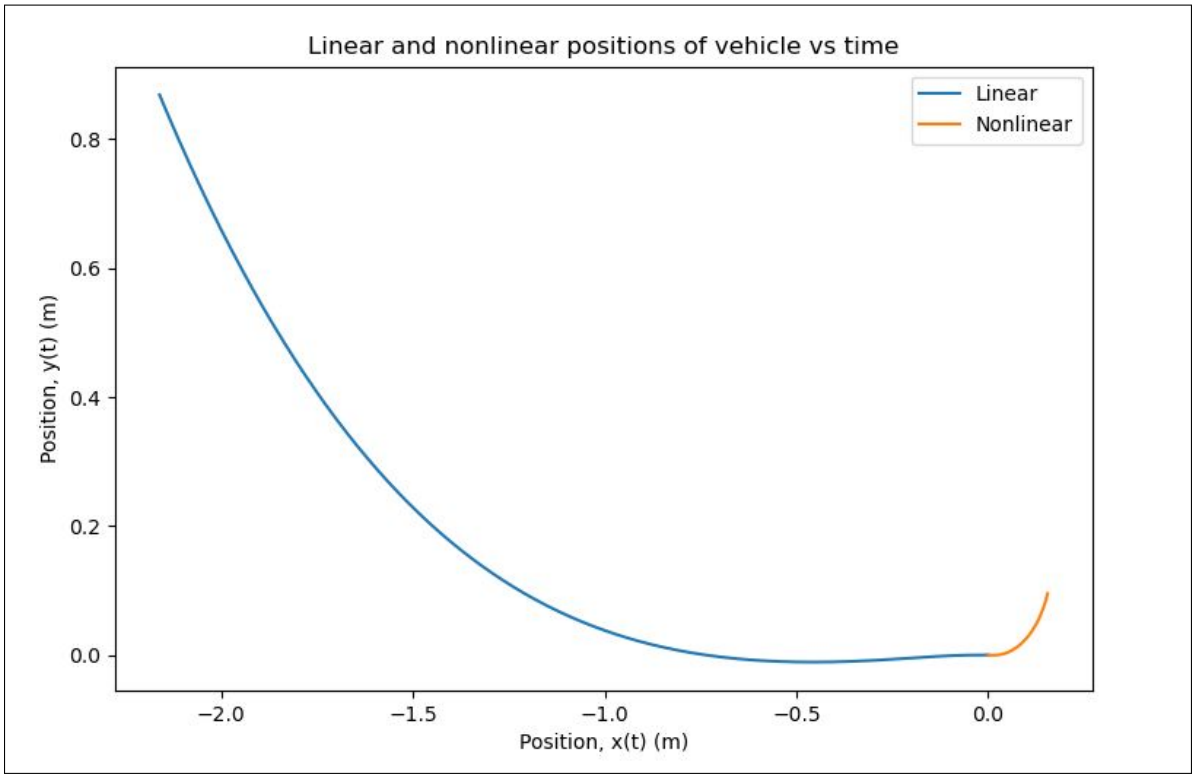

Figure 4.1.5: Nonlinear and linear Y-positions vs X-positions using control input set 1

When viewing figure 4.1.1, and when analysing the control inputs, it is clear that for the time period [3, 25], the right speed spins faster than the left wheel. This means that the robotic vehicle will turn right for the first 2 seconds before turning left for the remainder of the simulation. This is shown in figure 4.1.3 as the y-position of the vehicle is positive for most of the time period and the left side of the

robotic vehicle is the positive y-direction. Additionally both control inputs are much slower than the equilibrium point. This implies that the linear model moves in the negative-x direction as $u_L = u_L - \pi$ and $u_R = u_R - \pi$ and since both u_L and u_R are both smaller than π, the linear model sees theses inputs as negative. This is shown in figure 4.1.2. Finally, θ increases as the y-position moves in the positive direction and decreases as it moves in the negative direction. This is shown in figure 4.1.4 as θ decreases for the first 2 seconds when the robotic vehicle is turning right and then increases as it changes course and starts turning left.

As expected the linear model performs very similarly to the nonlinear model near the linearization point (t = 0) and performs less accurately as one moves further away from this point.

Input 2

$u_L = 1.5\pi$

$u_R = \pi + \frac{t}{5}$

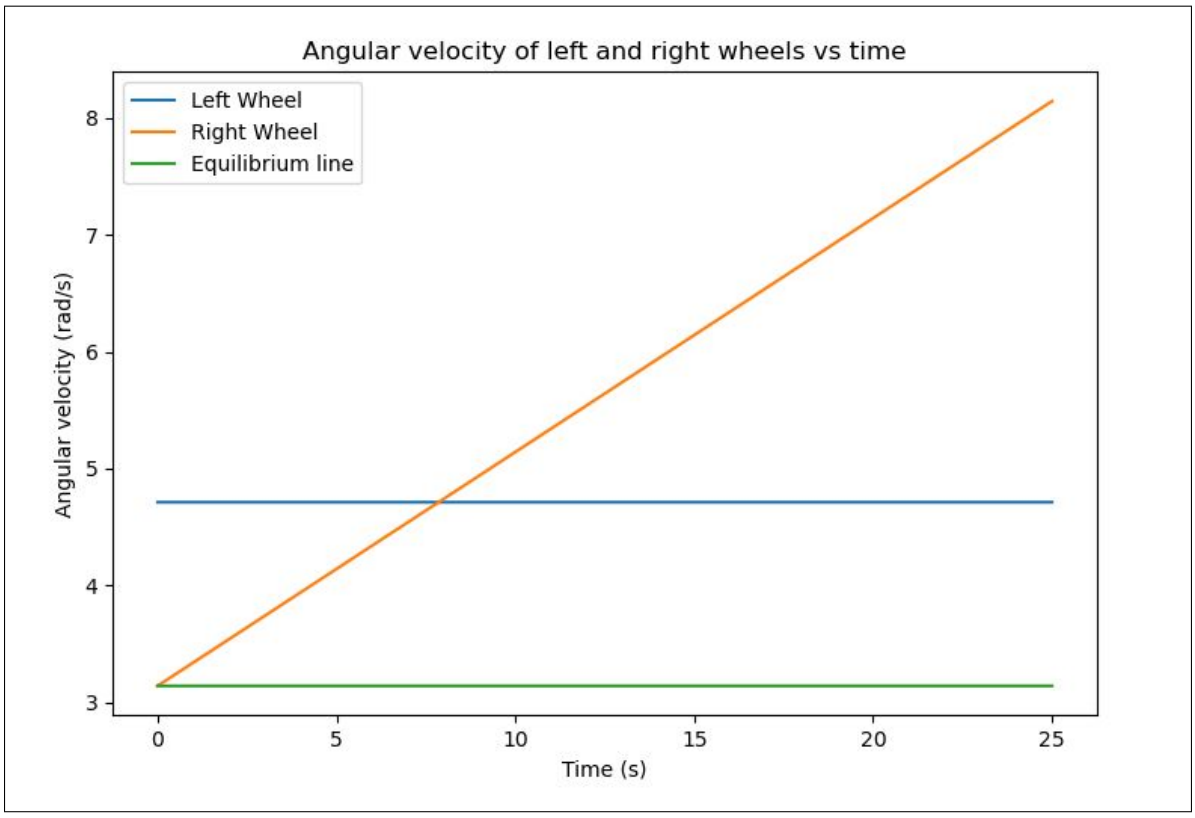

Figure 4.1.6: Angular speed of left wheel and right wheel vs time using control input set 2

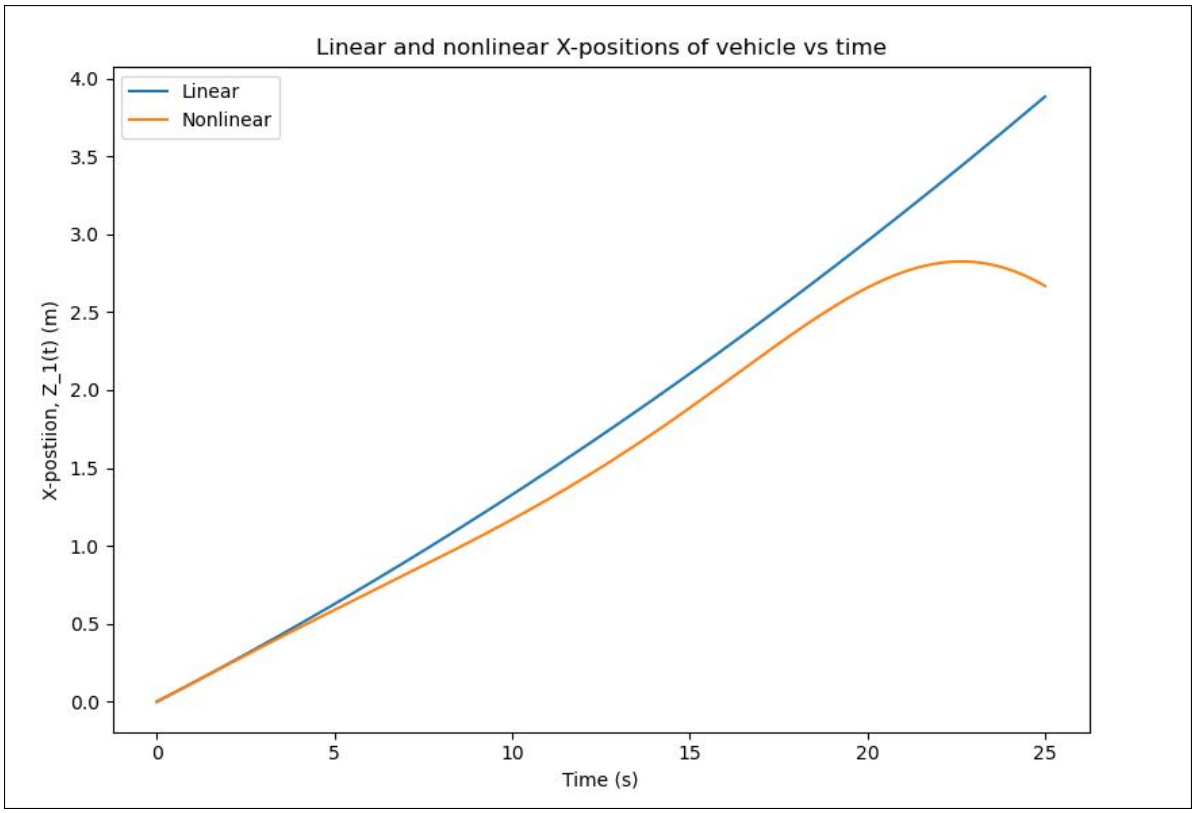

Figure 4.1.7: Nonlinear and linear X-positions vs time using control input set 2

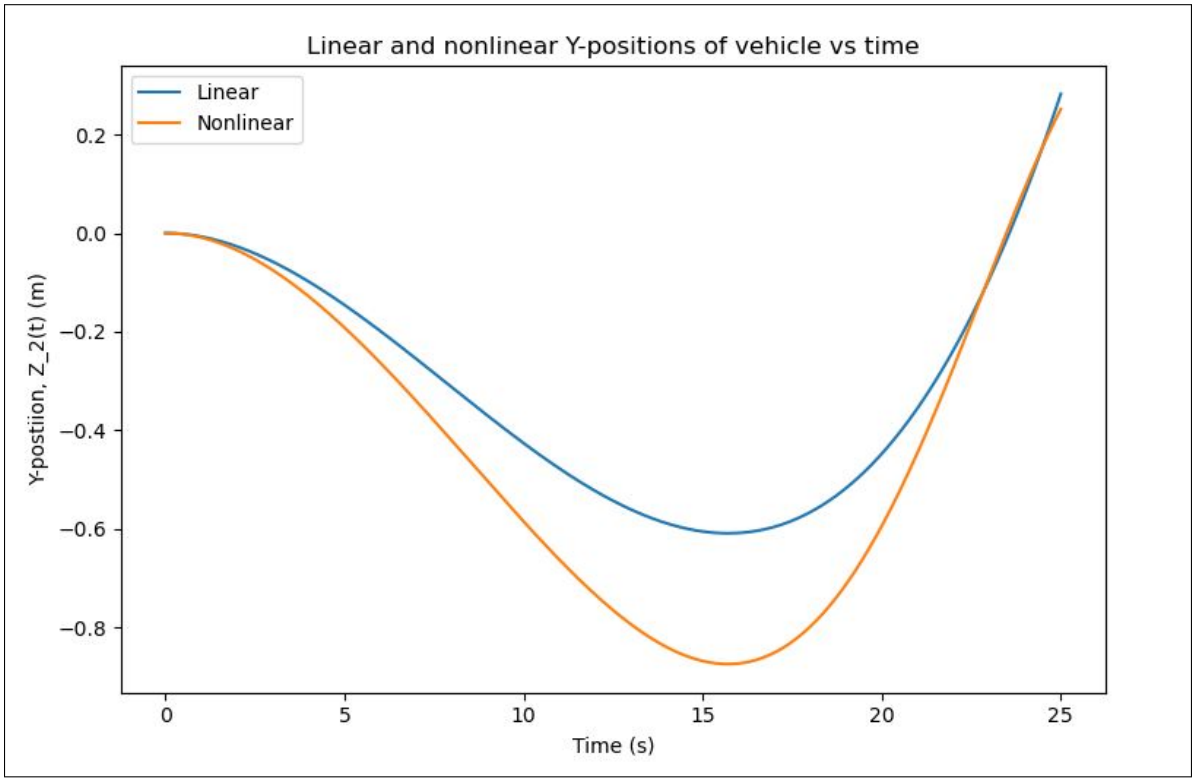

Figure 4.1.8: Nonlinear and linear Y-positions vs time using control input set 2

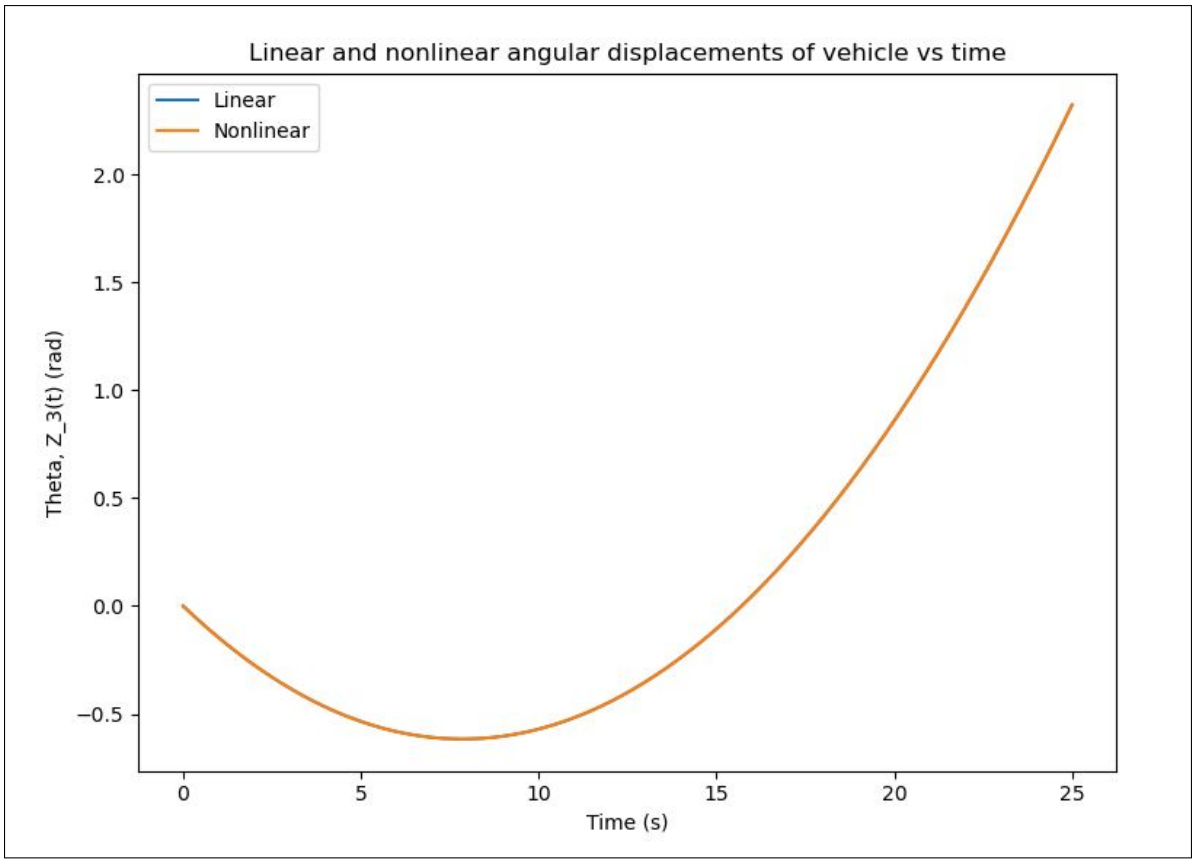

Figure 4.1.9: Nonlinear and linear angular displacements vs time using control input set 2

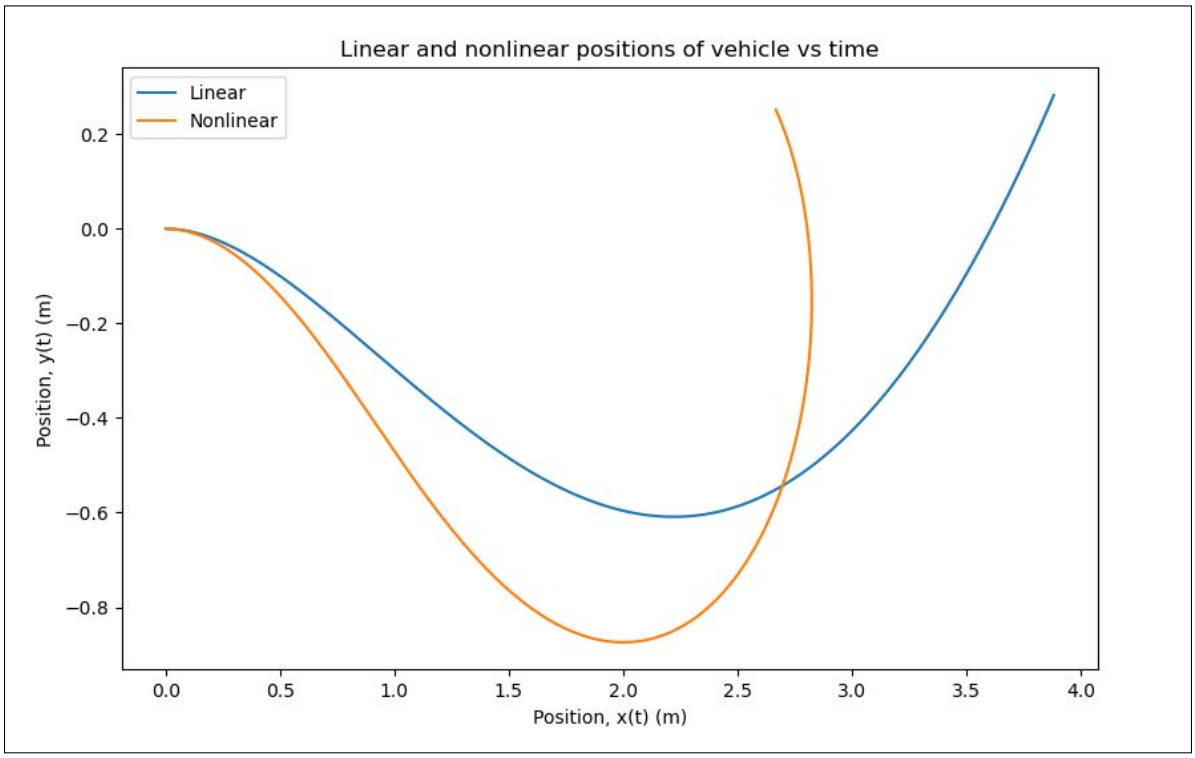

Figure 4.1.10: Nonlinear and linear Y-positions vs X-positions using control input set 2

When analysing figure 4.1.6 it is clear that both inputs are higher than the equilibrium, meaning that at all times the robotic vehicle will move in the positive x-direction. This shown in figure 4.1.7. $u_R > u_L$ for all $t > 8$. Therefore the robotic vehicle will turn right for the first 8 seconds before changing

direction turning left for the rest of the simulation. This shown in figure 4.1.8 where the y-position heads towards the negative direction (right side of the vehicle), before changing direction and moving in the positive direction. The vehicle only moves in the positive direction after 15 seconds, however this is because previous rotation towards the right had to be corrected before the vehicle moves towards the left. Figure 4.1.9 shows an expected results with θ decreasing when the vehicle turns right (time values less than 8s in this case) and increases for all time values where the vehicle turns left (t > 8).

The linear model has the same values as the nonlinear model for time values close to 0 (the equilibrium point) and performs less well as the time value increases. This is shown in all figures from input set 2.

4.2 Task 4

The following simulations were done with the control inputs:

u_L :

$$\begin{cases} \pi + 0.16 & t < 7 \\ \pi + 0.15 & 7 \leq t \leq 18 \\ \pi + 0.35 & t > 18 \end{cases}$$

u_R :

$$\begin{cases} \pi + 0.35 & t < 4 \\ \pi + 0.25 & 4 \leq t \leq 13 \\ \pi + 0.40 & t > 13 \end{cases}$$

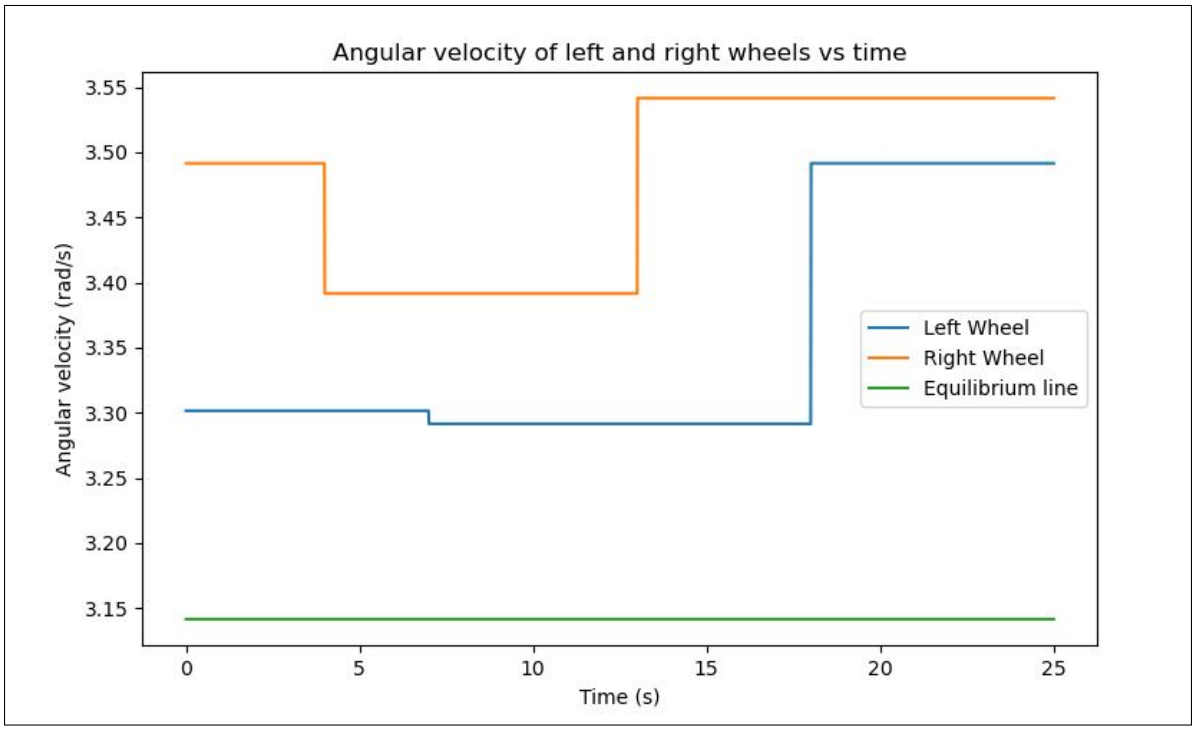

Figure 4.2.1: Angular speed of left wheel and right wheel vs time

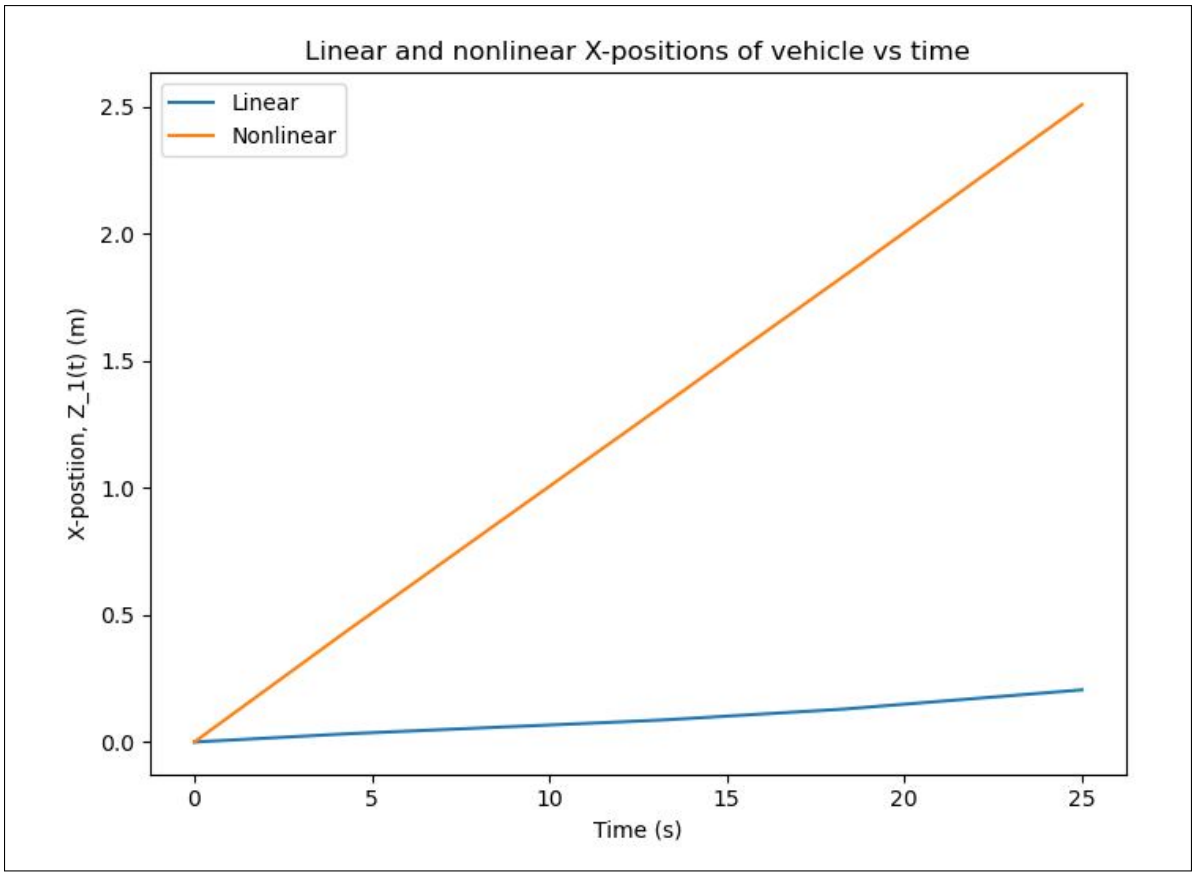

Figure 4.2.2: Nonlinear and linear X-positions vs time

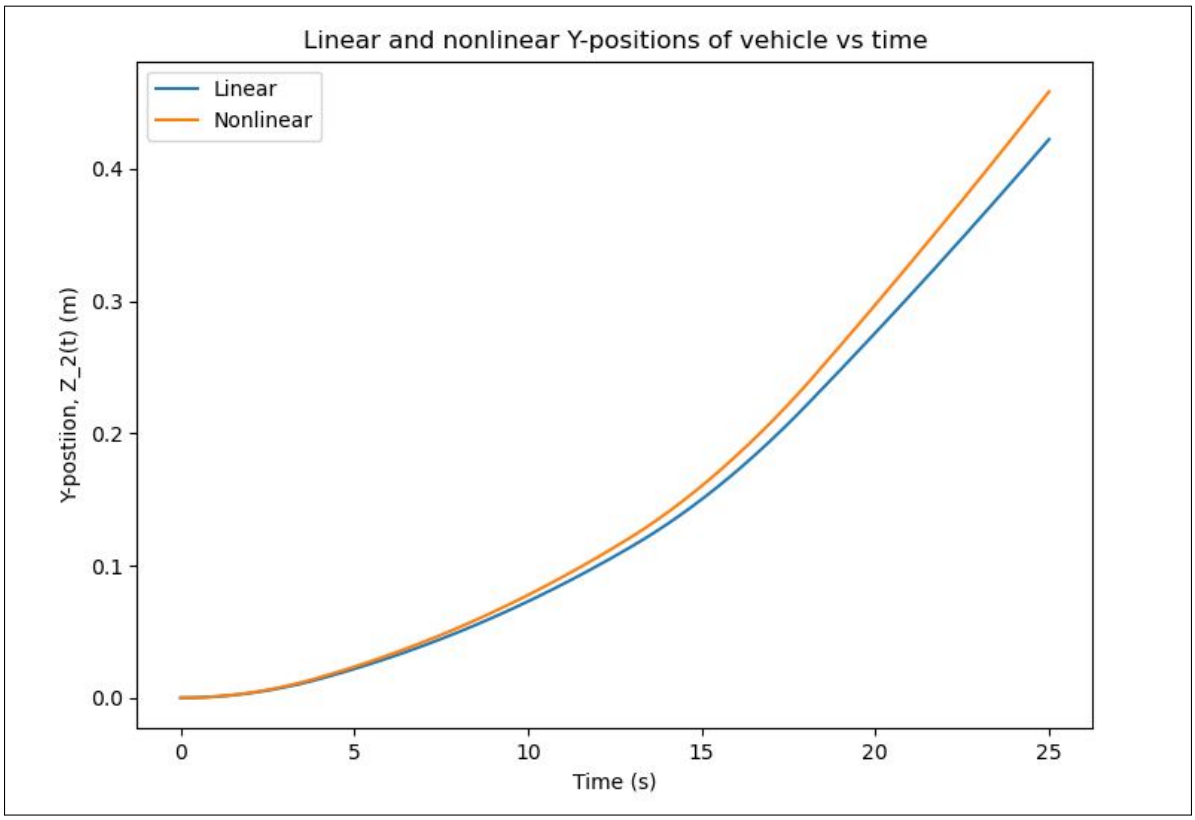

Figure 4.2.3: Nonlinear and linear Y-positions vs time

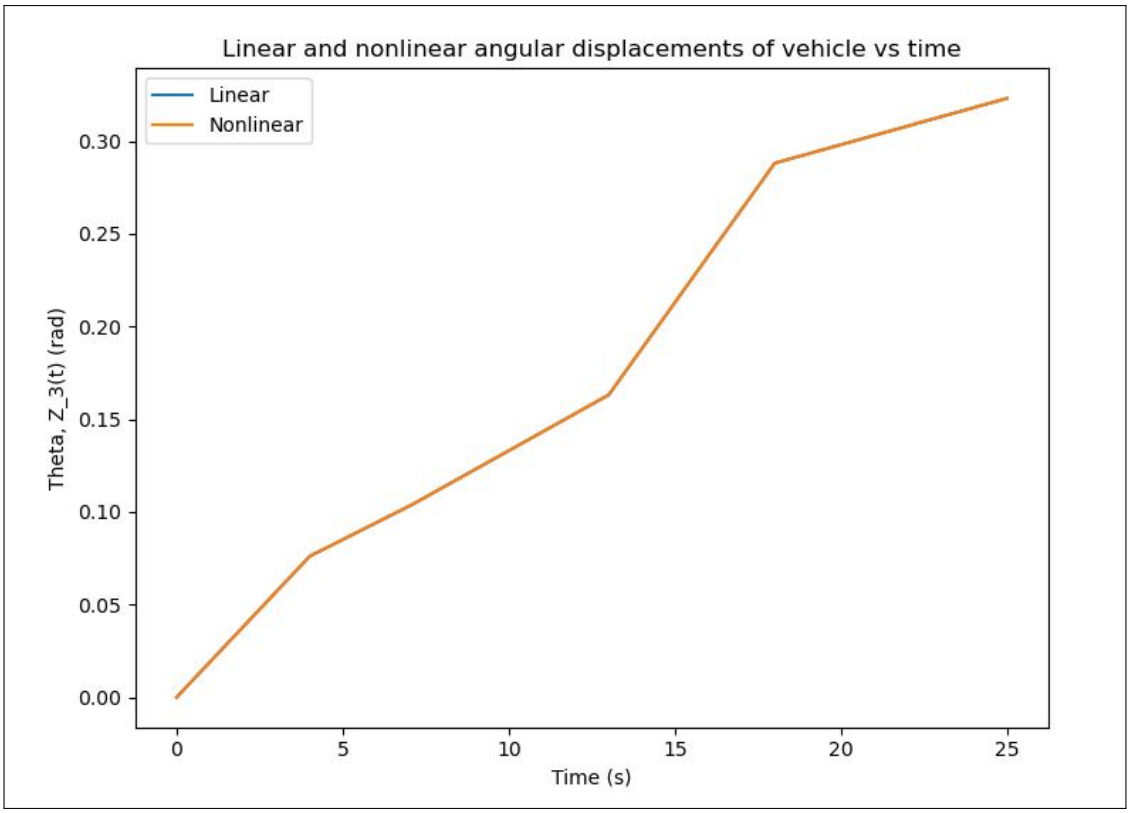

Figure 4.2.4: Nonlinear and linear angular displacements vs time

The angular displacement of the linear and nonlinear models is identical as can be seen in the figure above.

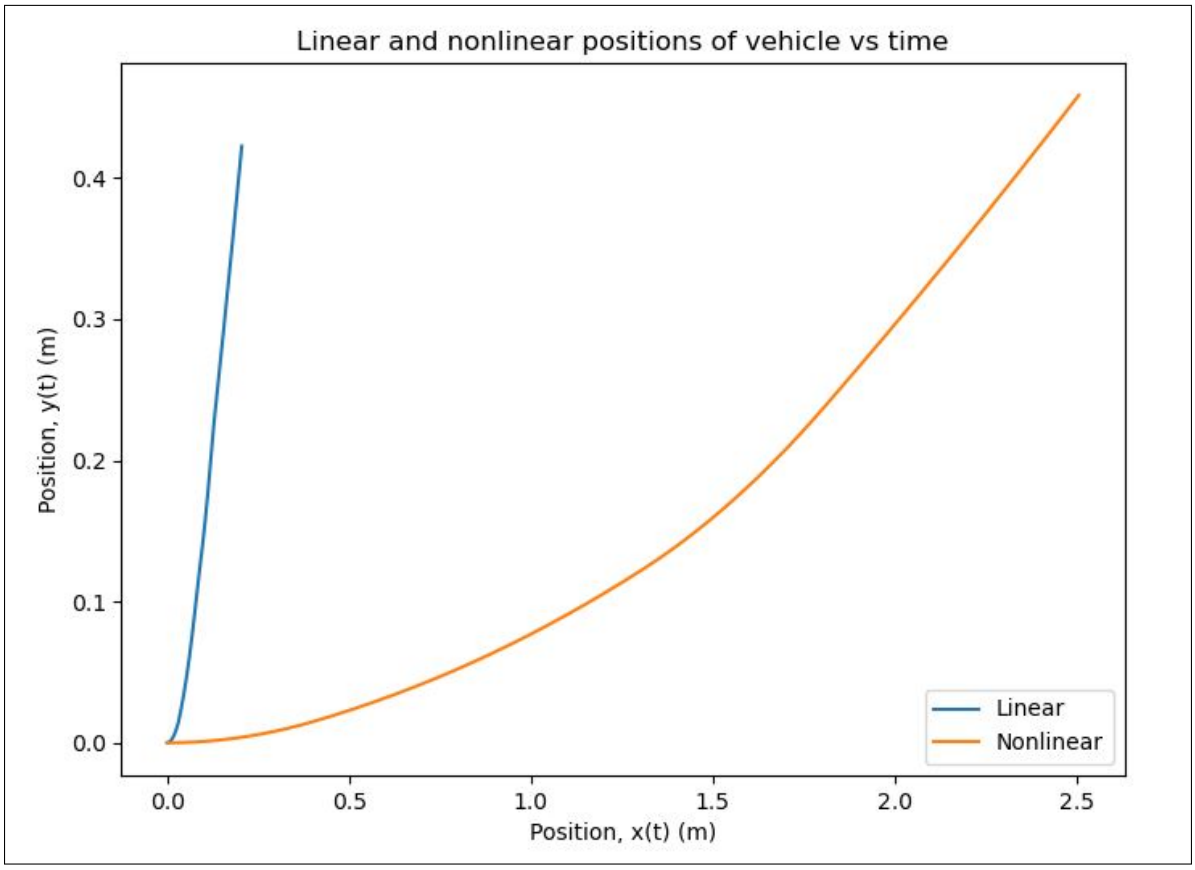

Figure 4.2.5: Nonlinear and linear Y-positions vs X-positions

Figure 4.2.1 shows the angular velocities of the left and right wheels, as well as the equilibrium point. When viewing this figure it is clear than the right wheel spins faster for time values, meaning that the robotic vehicle will only turn left. Additionally since both speeds are greater than the equilibrium speed, the linear x-positions will be positive, as can be seen in figure 4.2.2. The robotic vehicle should turn left for all time values. This result can be verified by observing figure 4.2.3, where the y-position increases meaning that the vehicle turns left (the positive y-direction is to the left of the robotic vehicle). Alternatively this result could be verified by viewing figure 4.2.4. θ increases as the vehicle turns left and when analysing said figure, θ only increases for all values of time. This implies that the robotic vehicle only turns left.

When comparing the linear and nonlinear models, it is clear that the linear model is a good approximation of the nonlinear for time values close to the equilibrium point (t = 0.), but a poor approximation for time values far away from the equilibrium point.

5 Conclusion

Successful linearization of a nonlinear system was carried out at its equilibrium points. The transfer function was successfully derived from the state space model. Differential equations and state space equations was successfully used to simulate linear and nonlinear systems with random control inputs and given inputs.

Linearization is necessary because the rotational speed of each MARV wheel needs to be modeled as linear displacement and angle change. The linearization of the system has a great influence on the accuracy of the model, especially at points further away from the equilibrium points. It can be seen that the linearized system model responds faster to the input and the establishment time is shorter. It is obvious why linearization is useful when modeling nonlinear systems, because small changes around the equilibrium point can be modeled relatively easily. The stability of the system can also be seen from the figure above. The system continuously reaches the required set point and does not oscillate over time.

Bibliography

[1] B. Pshenichnyj, "The linearization method," *Optimization*, vol. 18, no. 2, pp. 179–196, 1987.

Appendix A

Python Code

```
# Author: Timothy Fogwill
# EBB 320 practical 1
# Group 28
# Date started: 2021/08/31
# Last Update: 2021/09/03

import numpy as np
import math
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# Paramter set B
#################
R = 0.03
L = 0.3

def u_l(t):
    if t < 7:
        return np.pi + 0.16
    elif (t >= 7) and (t <= 18):
        return np.pi + 0.15
    elif t > 18:
        return np.pi + 0.35


def u_r(t):
    if t < 4:
        return np.pi + 0.35
    elif (t >= 4) and (t <= 13):
        return np.pi + 0.25
    elif t > 13:
        return np.pi + 0.40
#################

# determines which control inputs are used.
control_input = 2
```

```
u_max = 2 * np.pi
u_e = [0.5 * u_max, 0.5 * u_max]

z_e = [0, 0, 0]


def u_l_custom(t):
    return np.sin(t) * math.pow(math.e, -t)


def u_r_custom(t):
    return t/25


def u_l_custom_2(t):
    return 1.5*np.pi


def u_r_custom_2(t):
    return np.pi + t/5

# Nonlinear
#################
def f_nonlinear(theta, u_1, u_2):
    dx = R/2*(u_1 + u_2) * np.cos(theta)
    dy = R/2*(u_1 + u_2) * np.sin(theta)
    dz = R/L*(u_2 - u_1)

    return [dx, dy, dz]


# In different format so that scipy.integrate.odeint cant be used
def f_nonlinear_alt(z, t):
    if control_input == 0:
        u_1 = u_l(t)
        u_2 = u_r(t)
    elif control_input == 1:
        u_1 = u_l_custom(t)
        u_2 = u_r_custom(t)
    else:
        u_1 = u_l_custom_2(t)
        u_2 = u_r_custom_2(t)


    dx = R/2*(u_1 + u_2) * np.cos(z[2])
    dy = R/2*(u_1 + u_2) * np.sin(z[2])
    dz = R/L*(u_2 - u_1)

    return [dx, dy, dz]
#################

# Linear
#################
```

```python
def f_linear(theta, u1, u2):
    u_1 = u1 - u_e[0]
    u_2 = u2 - u_e[1]

    # Subbing in intitial conditions yields first half of taylor series formula.
    # Solving linear equation x = Ax + Bu yields second ha;f.

    dx = R/2 * (u_e[0] - u_e[1]) * np.cos(z_e[2]) + R/2 * (u_1 + u_2)
    dy = R/2 * (u_e[0] - u_e[1]) * np.sin(z_e[2]) + R*np.pi*theta
    dz = R/L * (u_e[1] - u_e[0]) + R/L * (u_2 - u_1)

    return [dx, dy, dz]


# In different format so that scipy.integrate.odeint cant be used
def f_linear_alt(z, t):
    if control_input == 0:
        u_1 = u_l(t) - u_e[0]
        u_2 = u_r(t) - u_e[1]
    elif control_input == 1:
        u_1 = u_l_custom(t) - u_e[0]
        u_2 = u_r_custom(t) - u_e[1]
    else:
        u_1 = u_l_custom_2(t)
        u_2 = u_r_custom_2(t)


    dx = R/2 * (u_e[0] - u_e[1]) * np.cos(z_e[2]) + R/2 * (u_1 + u_2)
    dy = R/2 * (u_e[0] - u_e[1]) * np.sin(z_e[2]) + R*np.pi*z[2]
    dz = R/L * (u_e[1] - u_e[0]) + R/L * (u_2 - u_1)

    return [dx, dy, dz]
#################

def Plot(control):
    t = np.arange(0, 25.01, 0.01)

    u_left = []
    u_right = []
    equi_line = []

    if control_input == 0:
        for i in range(len(t)):
            u_left.append(u_l(t[i]))
            u_right.append(u_r(t[i]))
            equi_line.append(np.pi)
    elif control_input == 1:
        for i in range(len(t)):
            u_left.append(u_l_custom(t[i]))
            u_right.append(u_r_custom(t[i]))
            equi_line.append(np.pi)
    else:
        for i in range(len(t)):
            u_left.append(u_l_custom_2(t[i]))
```

```
        u_right.append(u_r_custom_2(t[i]))
        equi_line.append(np.pi)



# U(t) vs t
plt.title("Angular velocity of left and right wheels vs time")
plt.ylabel("Angular velocity (rad/s)")
plt.xlabel("Time (s)")
plt.plot(t, u_left, label = "Left Wheel")
plt.plot(t, u_right, label = "Right Wheel")
plt.plot(t, equi_line, label = "Equilibrium line")
plt.legend()
plt.show()


# Both methods should yield similar results - scipy.integrate and the taylor series
if control == 0:
    x_pos_lin = [0]
    y_pos_lin = [0]
    theta_val_lin = [0]

    x_pos_nonlin = [0]
    y_pos_nonlin = [0]
    theta_val_nonlin = [0]

    for i in range(len(t) - 1):
        hold = t[i+1] - t[i]

        x_pos_lin.append(x_pos_lin[i] +
                         f_linear(theta_val_lin[i], u_left[i], u_right[i])[0]*hold)
        y_pos_lin.append(y_pos_lin[i] +
                         f_linear(theta_val_lin[i], u_left[i], u_right[i])[1]*hold)
        theta_val_lin.append(theta_val_lin[i] +
                             f_linear(theta_val_lin[i], u_left[i], u_right[i])[2]*hold)

        x_pos_nonlin.append(x_pos_nonlin[i] + f_nonlinear(
            theta_val_nonlin[i], u_left[i], u_right[i])[0]*hold)
        y_pos_nonlin.append(y_pos_nonlin[i] + f_nonlinear(
            theta_val_nonlin[i], u_left[i], u_right[i])[1]*hold)
        theta_val_nonlin.append(theta_val_nonlin[i] + f_nonlinear(
            theta_val_nonlin[i], u_left[i], u_right[i])[2]*hold)
else:
    init = [0, 0, 0]
    lin_sol = odeint(f_linear_alt, init , t)
    nonlin_sol = odeint(f_nonlinear_alt, init, t)



    x_pos_lin = lin_sol[:,0]
    y_pos_lin = lin_sol[:,1]
    theta_val_lin = lin_sol[:,2]

    x_pos_nonlin = nonlin_sol[:,0]
```

```
    y_pos_nonlin = nonlin_sol[:,1]
    theta_val_nonlin = nonlin_sol[:,2]



    # z(t) vs t
    # X-pos
    plt.title("Linear and nonlinear X-positions of vehicle vs time")
    plt.ylabel("X-postiion, Z_1(t) (m)")
    plt.xlabel("Time (s)")
    plt.plot(t, x_pos_lin, label = "Linear")
    plt.plot(t, x_pos_nonlin, label = "Nonlinear")
    plt.legend()
    plt.show()


    # Y-pos
    plt.title("Linear and nonlinear Y-positions of vehicle vs time")
    plt.ylabel("Y-postiion, Z_2(t) (m)")
    plt.xlabel("Time (s)")
    plt.plot(t, y_pos_lin, label = "Linear")
    plt.plot(t, y_pos_nonlin, label = "Nonlinear")
    plt.legend()
    plt.show()

    # Theta
    plt.title("Linear and nonlinear angular displacements of vehicle vs time")
    plt.ylabel("Theta, Z_3(t) (rad)")
    plt.xlabel("Time (s)")
    plt.plot(t, theta_val_lin, label = "Linear")
    plt.plot(t, theta_val_nonlin, label = "Nonlinear")
    plt.legend()
    plt.show()

    # Z_1 vs Z_2

    plt.title("Linear and nonlinear positions of vehicle vs time")
    plt.ylabel("Position, y(t) (m)")
    plt.xlabel("Position, x(t) (m)")
    plt.plot(x_pos_lin, y_pos_lin, label = "Linear")
    plt.plot(x_pos_nonlin, y_pos_nonlin, label = "Nonlinear")
    plt.legend()
    plt.show()




Plot(1)
```